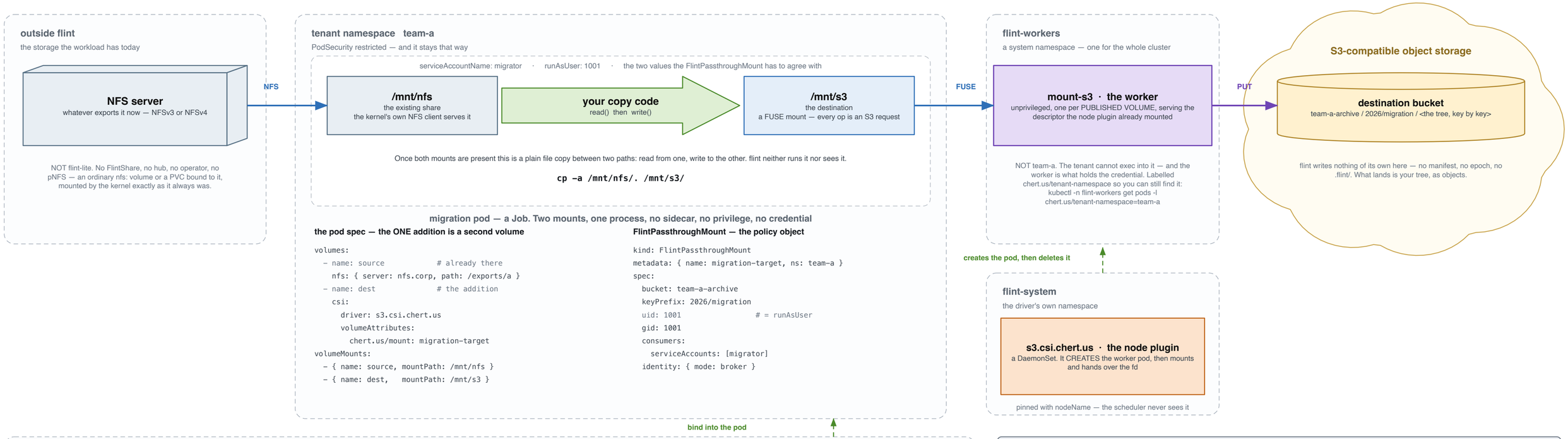
flint-passthrough

Migration — an NFS share into a bucket

The pod already mounts its NFS share. A SECOND volume in the same pod spec names a FlintPassthroughMount, and the CSI node plugin mounts the destination bucket at /mnt/s3. The copy itself is ordinary code in the pod — flint moves no bytes of its own.



s3.csi.chert.us — how /mnt/s3 gets there. There is NO webhook, and the pod is never mutated

One privileged process per node, holding no S3 credential and no Secrets RBAC. It is kubelet that calls it, at the moment the pod is scheduled — which is why nothing has to watch for a label.

1. **kubelet schedules the pod** — the csi: volume is inline — no PVC to bind, nothing to provision first
2. **NodePublishVolume** — kubelet passes a pod-bound SA token and its OWN csi.storage.k8s.io/* keys
3. **resolve the CR** — chert.us/mount names a FlintPassthroughMount in the TOKEN's namespace
4. **authorise the SA** — spec.consumers must list the pod's ServiceAccount. Absent means DENY
5. **mount(2) — ITSELF** — the plugin mounts, then passes /dev/fuse to a worker it created, over SCM_RIGHTS
6. **bind into the pod** — /mnt/s3 appears. The pod was never mutated: no webhook, no injected sidecar

there is no label, and the pod cannot name the bucket
chert.us/mount is domain-prefixed like a label, but it is a KEY in volumeAttributes on the inline csi: volume — not metadata.labels, and nothing admits or mutates on it. Bucket, keyPrefix, endpoint, region, image and credentials are refused BY NAME from the pod: only chert.us/mountIworkspace and chert.us/uidIgid are accepted. Kubelet's own csi.storage.k8s.io/* keys overwrite anything the pod puts there, and those are the only inputs the authorisation step trusts.

what bites a copy, specifically
uid — NodePublish never sees securityContext, so the CR's uid must match runAsUser or every write is EACCES · rsync — its default writes a temp file and renames, and Mountpoint has no rename: use cp, or rsync --inplace · readOnly on the destination CR is policy, not presentation, and refuses the copy outright.

THE POD IS CREATED THE ORDINARY WAY, and that is the whole point of the shape. You apply a Job whose spec has two volumes: the nfs: one it already had, and a csi: one naming a FlintPassthroughMount. There is no mutating webhook in flint at all, so nothing rewrites the pod between kubectl and kubelet — what you applied is what runs. The mount appears because kubelet calls NodePublishVolume on the node plugin when it schedules the pod, and the plugin does the one privileged act itself. A typo in chert.us/mount is therefore not an admission error but a FailedMount event naming the CR and the namespace, and the pod simply never starts.

IT USED TO BE A LABEL, and anyone who set this up before v1.45.0 will look for one. Until then a mutating webhook watched for flint.io/passthrough-mount on the pod — objectSelector, key Exists — and the label's VALUE named the CR, exactly as the volumeAttributes value does now. What it injected was a PRIVILEGED native sidecar, and that privilege was not reducible: /dev/fuse alone would take CAP_SYS_ADMIN and a hostPath device, but the mount has to REACH the app container, which needs mountPropagation: Bidirectional, which the API server permits only on a privileged container. So a namespace enforcing PodSecurity baseline or restricted REJECTED the mutated pod — correctly. `fcac038f` replaced both sidecar-injection webhooks (passthrough's and lean's) with the node DaemonSet, because spec.volumes[*].csi is on the restricted allow-list. The information you supply did not change; its carrier did.

THE WORKER IS A POD, AND IT IS NOT IN YOUR NAMESPACE. It lives in flint-workers — one system namespace for the whole cluster, not one per tenant — created by the node plugin during NodePublishVolume, pinned to this node with nodeName so the scheduler never places it, and owned by the Node object so a vanished node garbage-collects it. Three reasons it is not in team-a: the worker is what holds the credential, and anyone who can exec into pods in their own namespace could read it; PodSecurity is enforced differently, since flint-workers must be labelled privileged (the lean tree needs hostPath, forbidden under baseline) while your namespace stays restricted; and the plugin's own RBAC is a Role in flint-workers ALONE — pods create/get/list/watch/delete there and nowhere else, with a ValidatingAdmissionPolicy that further pins spec.nodeName to the node making the call, so a compromised node agent cannot place a pod anywhere but on itself. You can still find yours: chert.us/tenant-namespace and chert.us/tenant-pod are labels on the worker.

THE POD NAMES A CR; THE CR NAMES THE BUCKET. That indirection is the security property worth drawing, because it is what makes the destination a decision the platform owns rather than one the workload asserts. The pod author cannot point a mount at an arbitrary bucket by editing their own manifest — volumeAttributes are attacker-controlled input to a privileged process, and every key but the selector and the two integers is refused by name. Whoever may create a FlintPassthroughMount in that namespace decides which buckets exist to be named, and spec.consumers decides which ServiceAccounts may name them.

THE COPY IS YOUR CODE, running between two paths that happen to be backed by very different things: /mnt/nfs is served by the kernel's own NFS client and /mnt/s3 by a FUSE mount whose every operation is an S3 request. Nothing coordinates them and nothing needs to — the process reads a file and writes a file. What that costs is the S3 round trip in the copy's critical path, so throughput comes from concurrency: several files in flight, or several pods each given a subtree. What it saves is everything else — no staging disk, no intermediate format, no second copy of the data.

A RE-RUN IS SAFE, because a key is idempotent: writing the same object again yields the same object. That makes a Job with a backoff the right shape and makes "did it finish?" answerable by listing both sides rather than by trusting an exit code — the destination is a bucket, so the census is a LIST and the comparison is name-and-size. What a re-run will NOT do is repair a partially written object in place: Mountpoint writes each file once, sequentially, as a whole-object PUT, so an interrupted file is re-written from the beginning rather than resumed.

WHEN THIS IS THE WRONG TOOL. A passthrough mount is not POSIX — no rename, no append, no in-place modification — so it suits a copy and suits nothing that wants to keep editing afterwards. If the workload's own writes must continue against the destination with ordinary file semantics, the destination wants flint-lean (a local tree with a durable path behind it) or flint-lite (an NFS server whose working set is a PVC). Migrating INTO one of those is a different picture: the destination mount is theirs, and the copy step is the same plain code.

data plane — the bytes the copy moves

the object path — the same bytes, as requests

control plane — how the mount arrives

What the abbreviations mean

CSI ephemeral inline — a volume declared in the pod spec itself — no PVC, no PV, nothing to bind beforehand

volumeAttributes — the map on that inline volume. It is where chert.us/mount goes — NOT metadata.labels

NodePublishVolume — the CSI call kubelet makes when it schedules the pod. This is when the mount happens

FlintPassthroughMount — the CR that names the bucket, prefix, uid and consumers. The POLICY object

chert.us/mount — the only key that selects a destination. It names a CR in the pod's own namespace, never a bucket

spec.consumers — the ServiceAccounts a mount may be used by. ABSENT MEANS DENY, never means everyone

mount(2) — the syscall that attaches a filesystem to a path — the one act here that genuinely needs privilege

SCM_RIGHTS — the Unix-socket message that passes an open file descriptor from one process to another

FUSE — filesystem in userspace: the kernel hands each file operation to a process instead of a disk

mount-s3 — Mountpoint for S3, unchanged upstream. It serves the descriptor the node plugin already mounted

kernel NFS client — what serves /mnt/nfs. In-tree, nothing to do with flint — the share is mounted as it always was

flint-s3-broker — the one Deployment holding a standing credential. It issues short-lived keys to the worker only

EACCES on every write — what a uid mismatch looks like. The CR's uid must match the pod's runAsUser

no rename — Mountpoint cannot rename. rsync's default temp-file-then-rename fails; cp and rsync --inplace do not

whole-object PUT — each file is written once, sequentially. There is no append and no in-place modification

Job, not Deployment — a copy that finishes should be a Job — and a re-run is safe, because keys are idempotent